SEI On Load integration (display 22904)

What an On Load integration is

An On Load integration is a server-side script that Grow runs when an experience page opens. It reads URL parameters, can call external APIs, and returns values that Grow writes into form fields before the user types anything.

For SEI it does three things:

1. Decrypts the user's uid and email from the email link.
2. Reads that person from ZMP (`sei-dev`).
3. Prefills name, date of birth, both preference groups, and `fromOnLoad` .

When it runs

The browser sends `POST /api/integration/d/onload` only when a query parameter name starts with `nc.ol` . SEI links use two parameters:

Param	Plaintext
`nc.ol_tok`	uid string
`nc.ol_tok2`	email string

Both are encrypted with AES-CBC. The key is the UTF-8 bytes of the 32-character hex string (not hex-decoded). A random 16-byte IV is prepended to the ciphertext, padding is PKCS7, and the result is standard Base64, then URL-encoded.

`nc.pf_*` parameters only prefill fields. They do not trigger On Load.

The response arrives after `page:ready` , usually within 0.1–2 seconds.

Steps

#	Step	What it does
1	`eval$` decode	URL-decodes `params.ol_tok` / `params.ol_tok2` , turns spaces back into `+` , strips whitespace, Base64-decodes to bytes. Also builds `keyBytes` from the AES key.
2	`eval$` split	Bytes 0–15 are the IV (`ivBytes` , `ivBytes2`); the rest is ciphertext (`cipherBytes` , `cipherBytes2`). Returns null if the payload is shorter than 17 bytes.
3	`eval$` decrypt	`EncryptUtils.setupCipherWithIV(DECRYPT_MODE, …)` produces `uid` and `mail` . Any failure returns `''` , so the run continues.
4	`when$` + `http$`	Runs only if `uid` is non-empty. Sends `GET https://api.zetaglobal.net/ver2/sei-dev/subscribers?uid=…` with ZMP Basic auth. The response is stored as `executionResults.json` .

#	Step	What it does
5	`eval$` map	Unwraps a list response to the first person. Reads the email contact, `first_name`, `last_name`, and `date_of_birth`, which is split into year, unpadded month, and day. Maps `pref_center_*` flags equal to `Y` into choice ids for `multioptin` (role) and `multioptin_01` (interests).
6	`return$`	Copies `executionResults` into `data`. `fromOnLoad` is `true` only when both a uid and an email exist.

`encryptionKey` must equal the key in `keyBytes`. `onlySubSteps: true` makes Grow run only the steps above. `logDebug: true` logs runner output.

The Groovy, readable

This is the same logic as `onload.json`, laid out for reading. You can't paste it into Grow: there every value has to be a one-line `${{-> … }.call()}` string, and each step can only read earlier results through `executionResults.*`. In the real file, every key in step 5 recomputes `person` and `props` separately.

```
// ── Step 1 · eval$ — decode the URL tokens into bytes ──────────────
// Email clients and browsers can turn '+' into ' '. Put it back before
// and after URL-decoding, drop stray whitespace, then Base64-decode.
encryptedUid = {
    try {
        if (params.ol_tok == null) return null
        def s = java.net.URLDecoder.decode(params.ol_tok.trim().replace(' ', '+'), 'UTF-8')
        return s.replace(' ', '+').replaceAll('[\\r\\n\\t]', '').decodeBase64()
    } catch (Exception e) {
        return null
    }
}.call()

encryptedMail = { /* same as above, using params.ol_tok2 */ }.call()

// AES key = UTF-8 bytes of the 32-char hex string (NOT hex-decoded).
// Must be identical to "encryptionKey" at the bottom of the JSON.
keyBytes = 'REPLACE_WITH_AES_KEY_HEX'.getBytes('UTF-8')


// ── Step 2 · eval$ — split IV and ciphertext ───────────────────────
// Token layout: [16-byte IV][ciphertext]. Under 17 bytes = nothing to decrypt.
def ok      = encryptedUid?.length >= 17
ivBytes     = ok ? Arrays.copyOfRange(encryptedUid, 0, 16) : null
cipherBytes = ok ? Arrays.copyOfRange(encryptedUid, 16, encryptedUid.length) : null
// ivBytes2 / cipherBytes2: same thing for encryptedMail


// ── Step 3 · eval$ — AES-CBC decrypt ───────────────────────────────
uid = {
    try {
        if (keyBytes == null || cipherBytes == null || ivBytes == null) return ''
        def cipher = wyn.common.utils.EncryptUtils.setupCipherWithIV(
            javax.crypto.Cipher.DECRYPT_MODE, keyBytes, ivBytes)
        return new String(cipher.doFinal(cipherBytes), 'UTF-8')
    } catch (Exception e) {
        // Wrong key or corrupted token: uid is '' and step 4 is skipped.
        // This fails silently; nothing is logged as an error.
        return ''
    }
}.call()

mail = { /* same as above, using cipherBytes2 / ivBytes2 */ }.call()
```

```
// ── Step 4 · when$ + http$ — read the person from ZMP ──────────────
// Not Groovy; this is the JSON step, shown as pseudocode:
//
//   when$  "${executionResults.uid}"         → skip the call if uid is ''
//   GET    https://api.zetaglobal.net/ver2/sei-dev/subscribers?uid=<uid>
//   Authorization: Basic base64("api:<ZMP REST key>")
//   httpResponseAsJsonVar$ "json"             → executionResults.json


// ── Step 5 · eval$ — map the ZMP person to form fields ─────────────
def raw    = executionResults.json
def person = raw instanceof List ? (raw ? raw[0] : null) : raw   // ZMP may wrap it in a list
def props  = (person instanceof Map && person.properties instanceof Map) ? person.properties : [:]

zmpEmail  = person?.contacts?.find { it.contact_type == 'email' }?.contact_value ?: person?.email ?: ''
firstName = props.first_name ?: person?.first_name ?: ''
lastName  = props.last_name  ?: person?.last_name  ?: ''

// ZMP date_of_birth looks like "1989-02-19T00:00:00Z"
def d    = (props.date_of_birth ?: '') as String
dob      = d
dobYear  = d.length() >= 4  ? d.substring(0, 4)  : ''
def mm   = d.length() >= 7  ? d.substring(5, 7)  : ''
dobMonth = mm ? String.valueOf(mm.toInteger()) : ''   // "02" → "2"; Month options are 1–12
dobDay   = d.length() >= 10 ? d.substring(8, 10) : ''

// pref_center_* == 'Y'  →  checkbox choice ids
roles = []                                    // multioptin ("I am a")
if (props.pref_center_parent      == 'Y') roles << 'parent'
if (props.pref_center_coach_admin == 'Y') roles << 'coachadmin'
if (props.pref_center_player      == 'Y') roles << 'player'
if (props.pref_center_fan         == 'Y') roles << 'fan'

interests = []                                // multioptin_01 ("My preferences")
if (props.pref_center_cleats_and_shoes           == 'Y') interests << 'cleats_and_shoes'
if (props.pref_center_fan_jerseys_and_gear       == 'Y') interests << 'fan_jerseys_and_gear'
if (props.pref_center_apparel                    == 'Y') interests << 'apparel'
if (props.pref_center_soccer_balls_and_equipment == 'Y') interests << 'soccer_balls_and_equipment'
if (props.pref_center_goalkeeper                 == 'Y') interests << 'goalkeeper'
if (props.pref_center_new_content_and_articles   == 'Y') interests << 'new_content_and_articles'
if (props.pref_center_deals_and_special_offers   == 'Y') interests << 'deals_and_special_offers'


// ── Step 6 · return$ — what the form receives (data.*) ─────────────
// uid, email (= mail ?: zmpEmail), name_Firstname, name_Lastname,
// date_of_birth, date_of_birth_Year / _Month / _Day,
// multioptin (roles), multioptin_01 (interests), and:
fromOnLoad = (uid && (mail || zmpEmail)) ? 'true' : 'false'
```

After the response

- Grow applies `response.data` only when it is non-empty. An empty `{}` means the runner stopped before `return$`.
- The name, DOB, and preference modules are conditional on `fromOnLoad`. While a conditional module is hidden, Grow stores one prefilled value per module. When there are several values (first and last name), the last one wins. `custom.html` re-applies `name_Firstname` and the month from the `/onload` response after the module opens.
- Custom JS shows or hides T&C and Sign up based on `fromOnLoad`. Custom JS never decrypts tokens.

Failure causes seen on 22904

Symptom	Cause
No `/onload` request in Network	No `nc.ol*` param in the URL (`nc.pf_*` only, or none). Or the integration is not saved on this display.
No request, or empty fields	URL typo `&?nc.ol_tok=` . The extra `?` changes the param name.
`/onload` returns 200 `{}`	An `http$` that read uid from `encryptedTokenMapping` (official `nc.ol` blob) stopped before `return$` . Seen with ZMP, httpbin, and the official sample shape. The fix was to decrypt in `eval$` first, then call ZMP.
`/onload` returns 503 "Failed to make onload integration request!"	Earlier Groovy shape combined with a bad URL. Also seen with an httpbin variant that had `requestType$` and an empty `params$` and no `encryptionKey` .
uid/email empty, no ZMP call	Wrong or empty AES key (one test pasted `''.getBytes`). Or `keyBytes` and `encryptionKey` differ. Or tokens were encrypted with a different key. Decrypt errors return `''` , so `when$` skips the GET silently.
Decrypt fails on some links	`+` became a space in transit. Step 1 replaces spaces with `+` . Hand-built URLs must still URL-encode `+` , `/` , and `=` .
Email filled, everything else blank	That ZMP person has no data. `sei.dev.pc.003` – `008` were email-only in later checks. Only 001 and 002 had name, DOB, and prefs.
Last name fills, first name blank	Hidden conditional name module keeps one prefilled value; `name_Lastname` overwrites `name_Firstname` . Handled in Custom JS.
Month blank	ZMP gives `02` ; the select options are `1` – `12` . Step 5 now strips the leading zero.
Nested `name` object or single `date_of_birth` string did not fill inputs	Grow needs the flat keys `name_Firstname` , `name_Lastname` , `date_of_birth_Year/Month/Day` .
Direct ZMP curl works but the form is empty	The failure is in Grow's runner, not ZMP. Check the `/onload` response body.

Debugging

1. Save the On Load JSON on the experience.
2. Open staging with `&nc.ol_tok=…&nc.ol_tok2=…` . Add `&debug=6` for event logs.
3. In DevTools → Network, open `/api/integration/d/onload` and read the response `data` .
4. Compare it with `GET /ver2/sei-dev/subscribers?uid=…` for the same user.

Test users: `clients/SEI-Soccer-9958/sei-dev-test-users.md` . Each new link needs fresh tokens encrypted with the current key. Old tokens keep working as long as the key is unchanged.